

Nature of Software – Defining Software

Software is:

- 1) Instructions (computer programs) that when executed provide desired features, function, and performance;*
- 2) Data structures that enable the programs to adequately manipulate information.*
- 3) Documentation that describes the operation and use of the programs.*

What is Software?

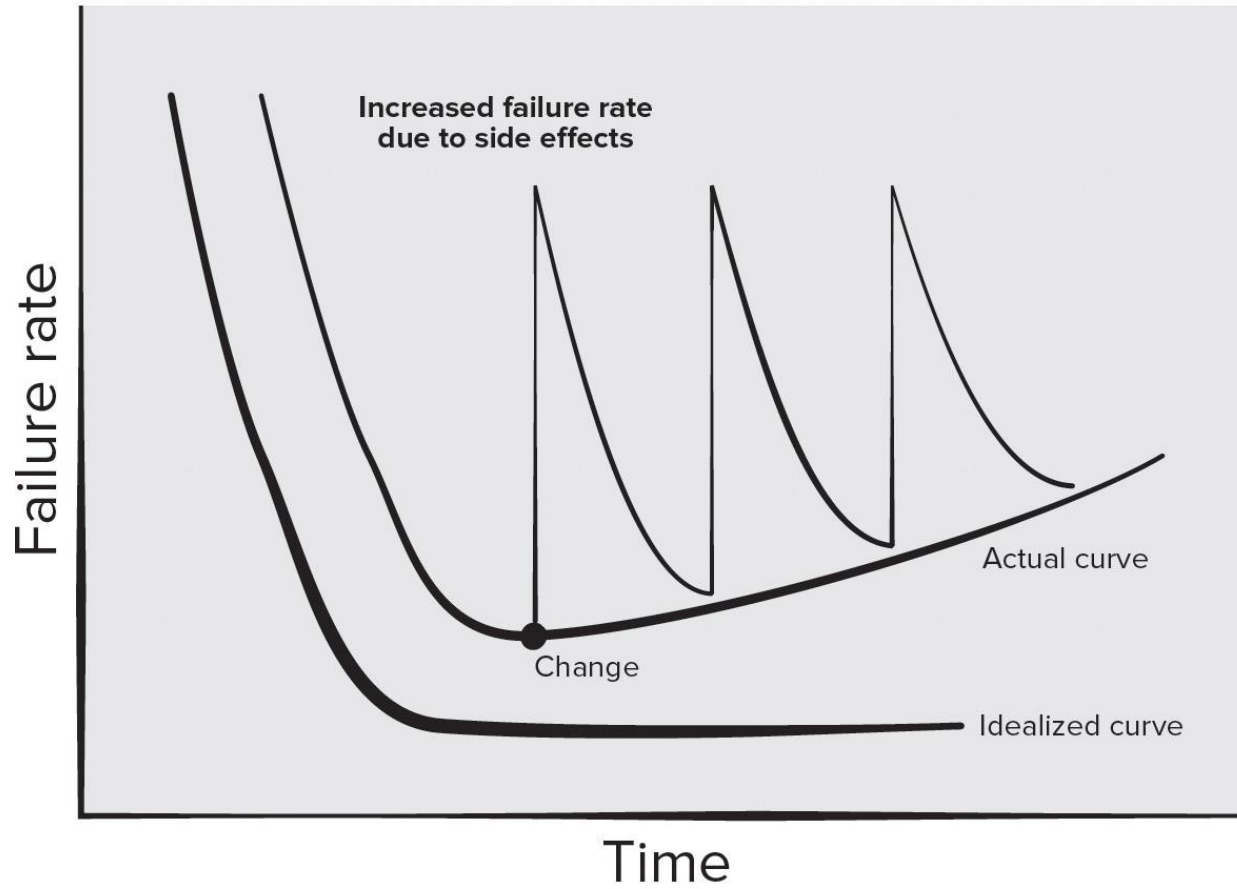
- *Software is developed or engineered it is not manufactured in the classical sense.*
- *Software doesn't "wear out" but it does deteriorate.*
- *Although the industry is moving toward component-based construction, most software continues to be custom-built.*

Software Application Domains

- System software.
- Application software.
- Engineering/Scientific software.
- Embedded software.
- Product-line software.
- Web/Mobile applications.
- AI software (robotics, neural nets, game playing).

Wear versus Deterioration

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Process Framework Activities

Communication.

Planning.

Modeling.

- Analysis of requirements.
- Design.

Construction:

- Code generation.
- Testing.

Deployment.

Hooker's General Principles

for software engineering practice

1. *The Reason It All Exists* – provide value to users.
2. *KISS (Keep It Simple, Stupid!)* – design simple as it can be.
3. *Maintain the Vision* – clear vision is essential.
4. *What You Produce, Others Will Consume.*
5. *Be Open to the Future* - do not design yourself into a corner.
6. *Plan Ahead for Reuse* – reduces cost and increases value.
7. *Think!* – placing thought before action produce results.

Identifying a Task Set

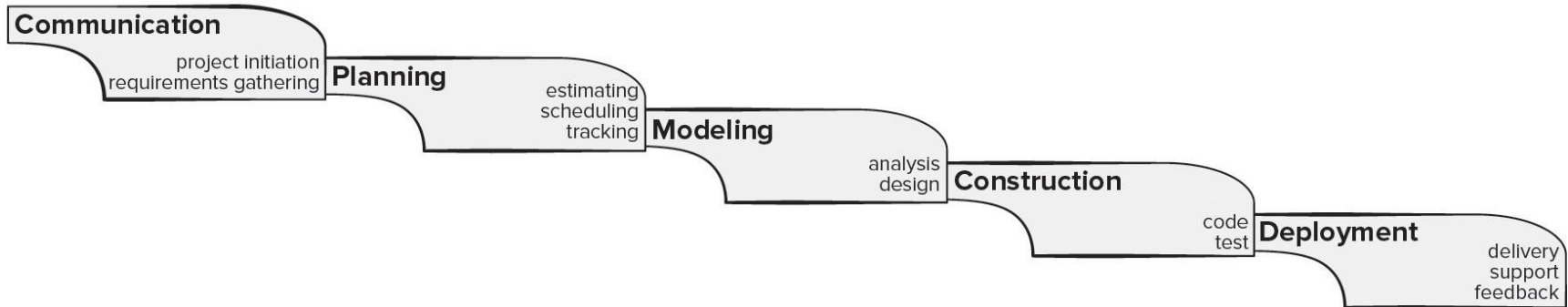
A task set defines the actual work to be done to accomplish the objectives of a software engineering action.

A task set is defined by creating several lists:

- A list of the tasks to be accomplished.
- A list of the work products to be produced.
- A list of the quality assurance filters to be applied.

Waterfall Process Model

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Pros

- It is easy to understand and plan.
- It works for well-understood small projects.
- Analysis and testing are straightforward.

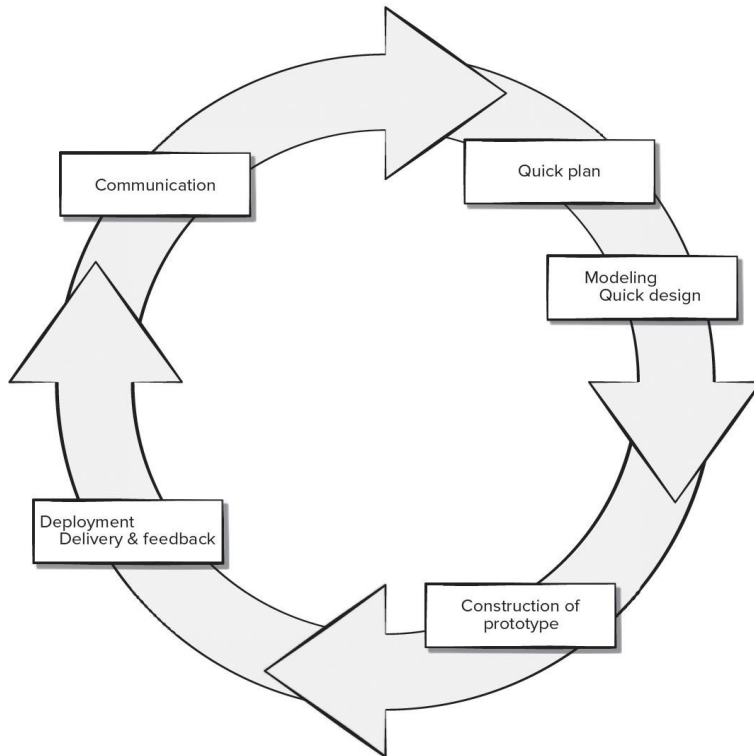
Cons

- It does not accommodate change well.
- Testing occurs late in the process.
- Customer approval is at the end.

[Access the text alternative for slide images.](#)

Prototyping Process Model

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Pros

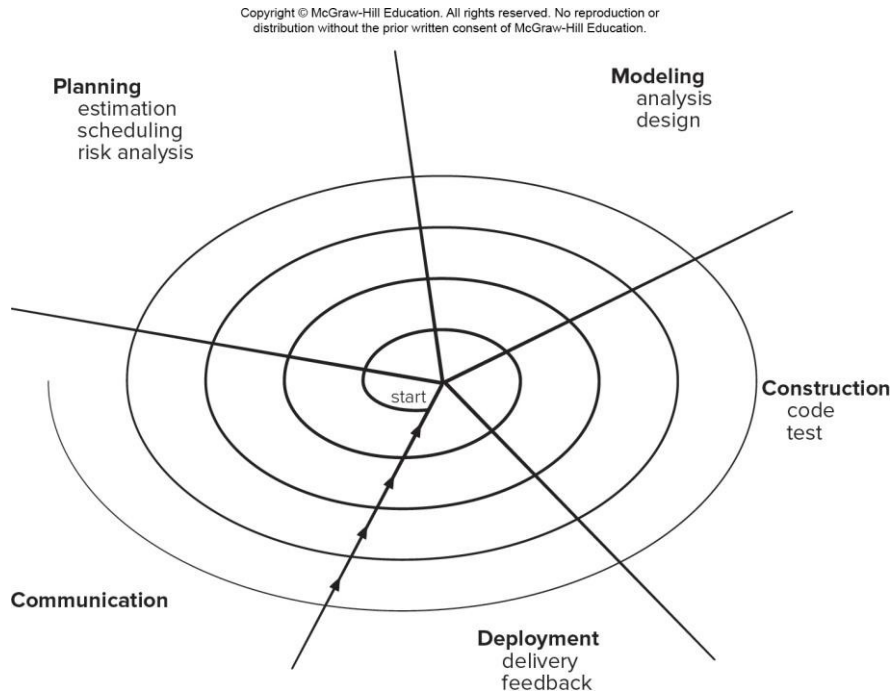
- Reduced impact of requirement changes.
- Customer is involved early and often.
- Works well for small projects.
- Reduced likelihood of product rejection.

Cons

- Customer involvement may cause delays.
- Temptation to “ship” a prototype.
- Work lost in a throwaway prototype.
- Hard to plan and manage.

[Access the text alternative for slide images.](#)

Spiral Process Model



Pros

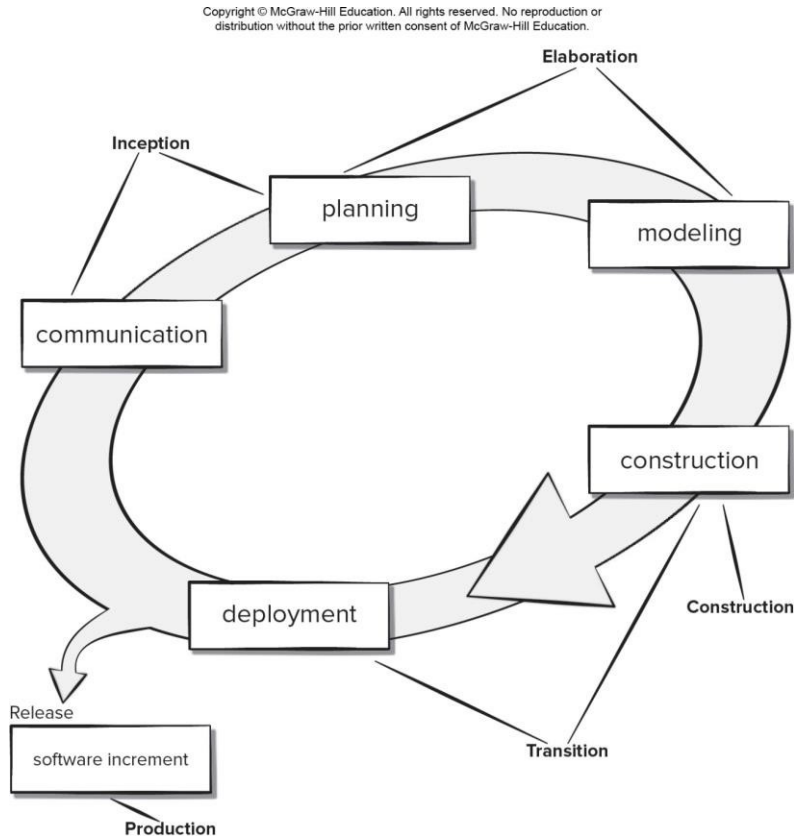
- Continuous customer involvement.
- Development risks are managed.
- Suitable for large, complex projects.
- It works well for extensible products.

Cons

- Risk analysis failures can doom the project.
- Project may be hard to manage.
- Requires an expert development team.

[Access the text alternative for slide images.](#)

Unified Process Model



Pros

- Quality documentation emphasized.
- Continuous customer involvement.
- Accommodates requirements changes.
- Works well for maintenance projects.

Cons

- Use cases are not always precise.
- Tricky software increment integration.
- Overlapping phases can cause problems.
- Requires expert development team.

[Access the text alternative for slide images.](#)

What is Agility?

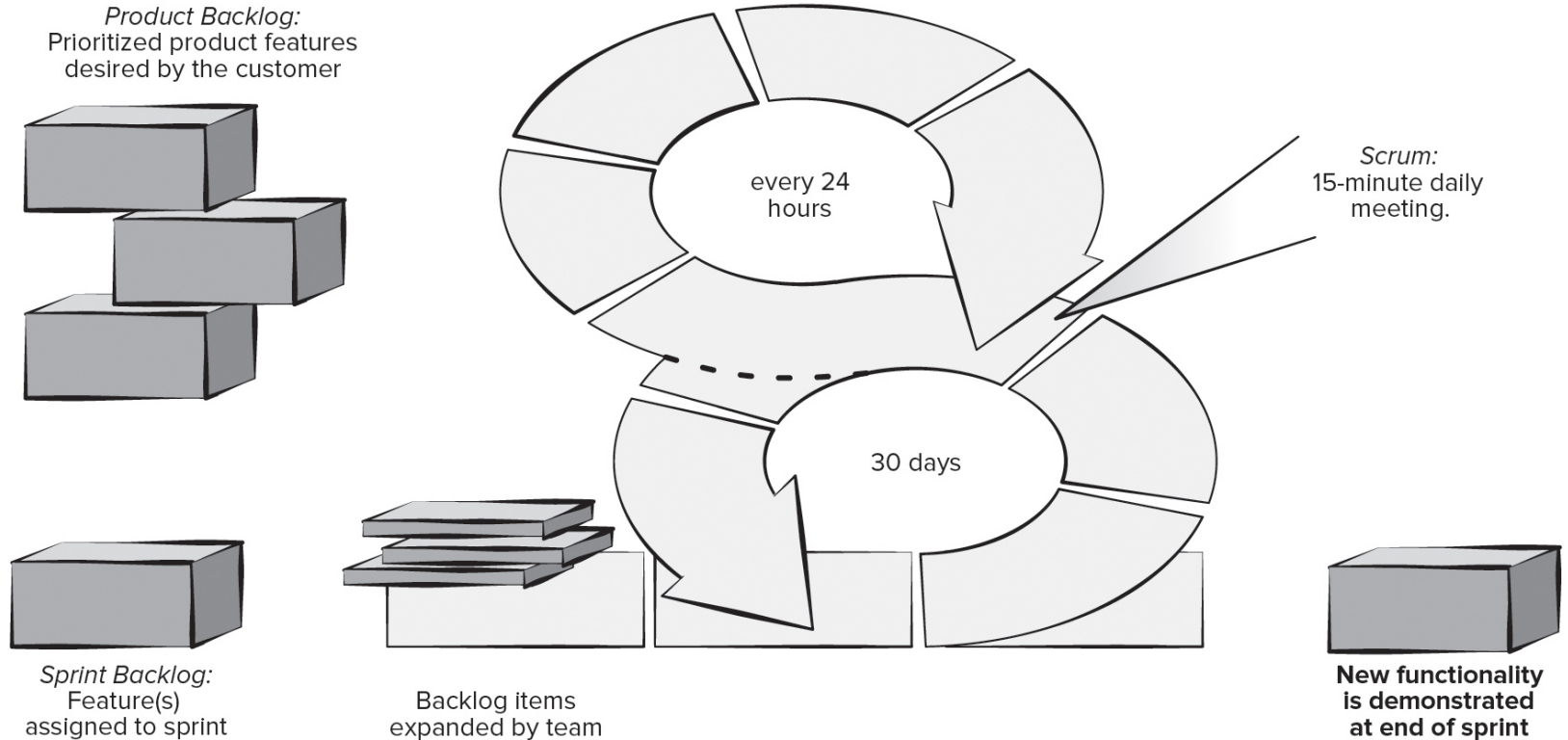
- Effective (rapid and adaptive) response to change.
- Effective communication among all stakeholders.
- Drawing the customer onto the team.
- Organizing a team so that it is in control of the work performed.
- Rapid, incremental delivery of software.

What is an Agile Process?

- Driven by customer descriptions of what is required (scenarios).
- Customer feedback is frequent and acted on.
- Recognizes that plans are short-lived.
- Develops software iteratively with a heavy emphasis on construction activities.
- Delivers multiple ‘software increments’ as executable prototypes.
- Adapts as project or technical changes occur.

Scrum Framework

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Scrum Details

- **Backlog Refinement Meeting** Developers work with stakeholders to create product backlog.
- **Sprint Planning Meeting** Backlog partitioned into “sprints” derived from backlog and next sprint defined.
- **Daily Scrum Meeting** Team members synchronize their activities and plan work day (15 minutes max).
- **Sprint Review** Prototype “demos” are delivered to the stakeholders for approval or rejection.
- **Sprint Retrospective** After sprint is complete, team considers what went well and what needs improvement.

Pros

- Product owner sets priorities.
- Team owns decision making.
- Documentation is lightweight.
- Supports frequent updating.

Cons

- Difficult to control the cost of changes.
- May not be suitable for large teams.
- Requires expert team members.

What is Agility?

- Effective (rapid and adaptive) response to change.
- Effective communication among all stakeholders.
- Drawing the customer onto the team.
- Organizing a team so that it is in control of the work performed.
- Rapid, incremental delivery of software.

Traits of Successful Software Engineers

- Sense of individual responsibility.
- Acutely aware of the needs of team members and stakeholders.
- Brutally honest about design flaws and offers constructive criticism.
- Resilient under pressure.
- Heightened sense of fairness.
- Attention to detail.
- Pragmatic adapting software engineering practices based on the circumstances at hand.

Effective Software Team Attributes

- Sense of purpose.
- Sense of involvement.
- Sense of trust.
- Sense of improvement.
- Diversity of team member skill sets.

Agile Teams

Stress individual competency coupled with group collaboration as critical success factors.

People trump process and politics can trump people.

Agile teams as self-organizing and have many structures.

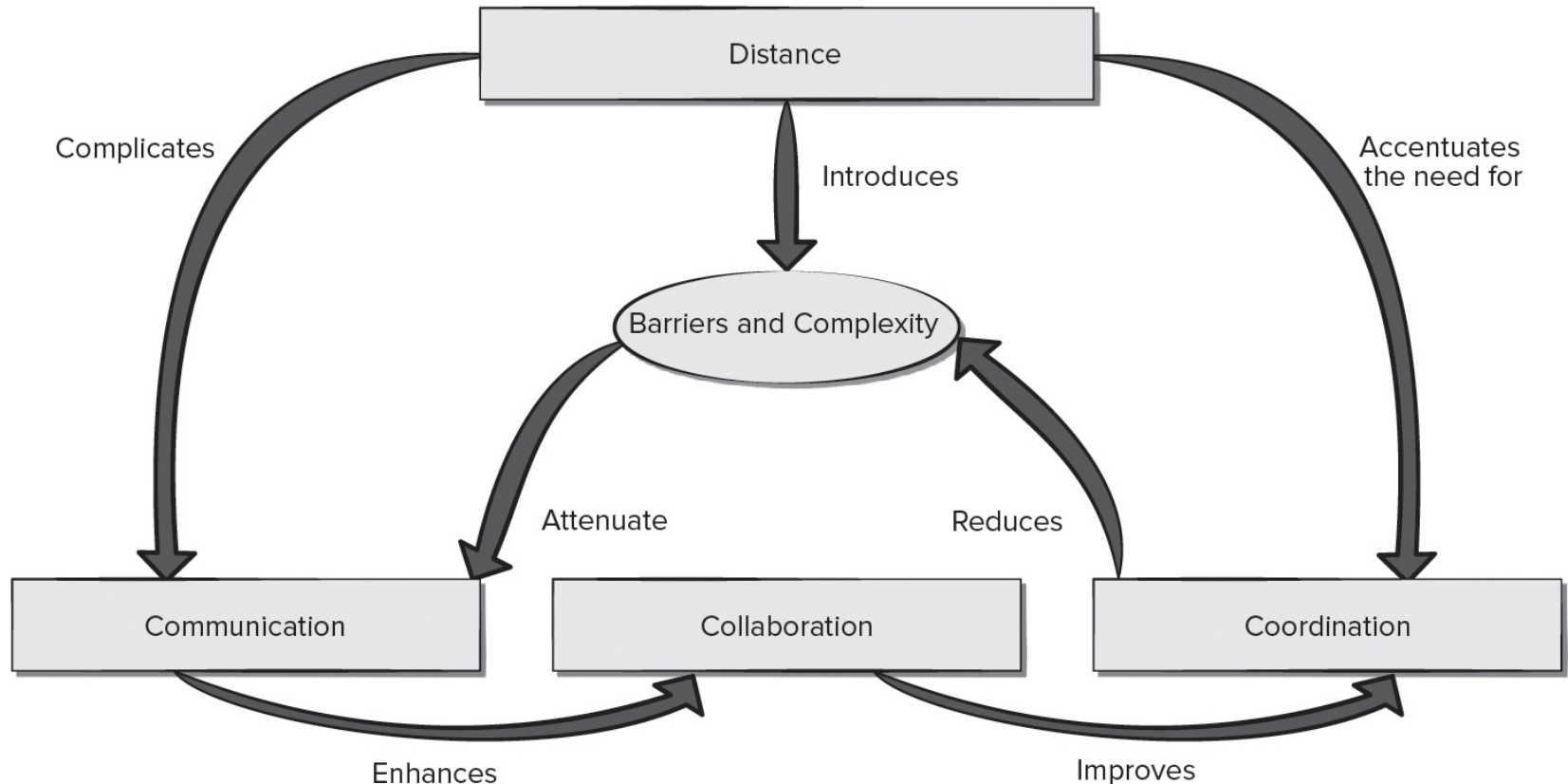
- Adaptive team structures.
- Significant autonomy.

Communication among developers and stakeholders is important (consider adding customer rep to team).

Planning is kept to a minimum and constrained only by business requirements and organizational standards.

Factors Affecting Global Software Development Teams

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Agile Modeling Principles ¹

- Principle #1. *The primary goal of the software team is to build software not create models.*
- Principle #2. *Travel light – don't create more models than you need.*
- Principle #3. *Strive to produce the simplest model that will describe the problem or the software.*
- Principle #4. *Build models in a way that makes them amenable to change.*
- Principle #5. *Be able to state an explicit purpose for each model that is created.*

Agile Modeling Principles ²

- **Principle #6. Adapt the models you create to the system at hand.**
- **Principle #7. Try to build useful models, forget about building perfect models.**
- **Principle #8. Don't become dogmatic about model syntax. Successful communication is key.**
- **Principle #9. If your instincts tell you a paper model isn't working you may have a reason to be concerned.**
- **Principle #10. Get feedback as soon as you can.**

Agile Testing Principles ₁

- **Principle #1.** *All tests should be traceable to customer requirements.*
- **Principle #2.** *Tests should be planned long before testing begins.*
 1. Testing is a process of executing a program with intent of finding an error,
 2. A good test case is one that has a high probability of finding an as-yet-undiscovered error.
 3. A successful test is one that uncovers an as-yet-undiscovered error.
- **Principle #3.** *The Pareto principle applies to software testing.*

Agile Testing Principles ²

- Principle #4. *Testing should begin “in the small” and progress toward testing “in the large.”*
- Principle #5. *Exhaustive testing is not possible.*
- Principle #6. *Testing effort for each system module commensurate to expected fault density.*
- Principle #7. *Static testing can yield high results.*
- Principle #8. *Track defects and look for patterns in defects uncovered by testing.*
- Principle #9. *Include test cases that demonstrate software is behaving correctly.*

Requirements Engineering ¹

- Inception - establish a basic understanding of the problem, the people who want a solution, and the nature of the solution that is desired, important to establish effective customer and developer communication.
- Elicitation - elicit requirements and business goals from all stakeholders.
- Elaboration - focuses on developing a refined requirements model that identifies aspects of software function, behavior, and information.

Requirements Engineering ²

- Negotiation—agree on the scope of a deliverable system that is realistic for developers and customers.
- Specification—can be any or all of the following: written documents, graphical models, mathematical models, usage scenarios, prototypes.
- Validation—Requirements engineering work products produced during requirements engineering are assessed for quality and consistency.
- Requirements management – set of traceability activities to help the project team identify, control, and track requirements and their changes to requirements as the project proceeds.

Non-functional Requirements

Non-Functional Requirement (NFR) – quality attribute, performance attribute, security attribute, or general system constraint.

A two-phase process is used to determine which NFR's are compatible:

- The first phase is to create a matrix using each NFR as a column heading and the system SE guidelines a row labels.
- The second phase is for the team to prioritize each NFR using a set of decision rules to decide which to implement by classifying each NFR and guideline pair as complementary, overlapping, conflicting, or independent.

Validating Requirements ₁

- Is each requirement consistent with the overall objective for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?

Validating Requirements ²

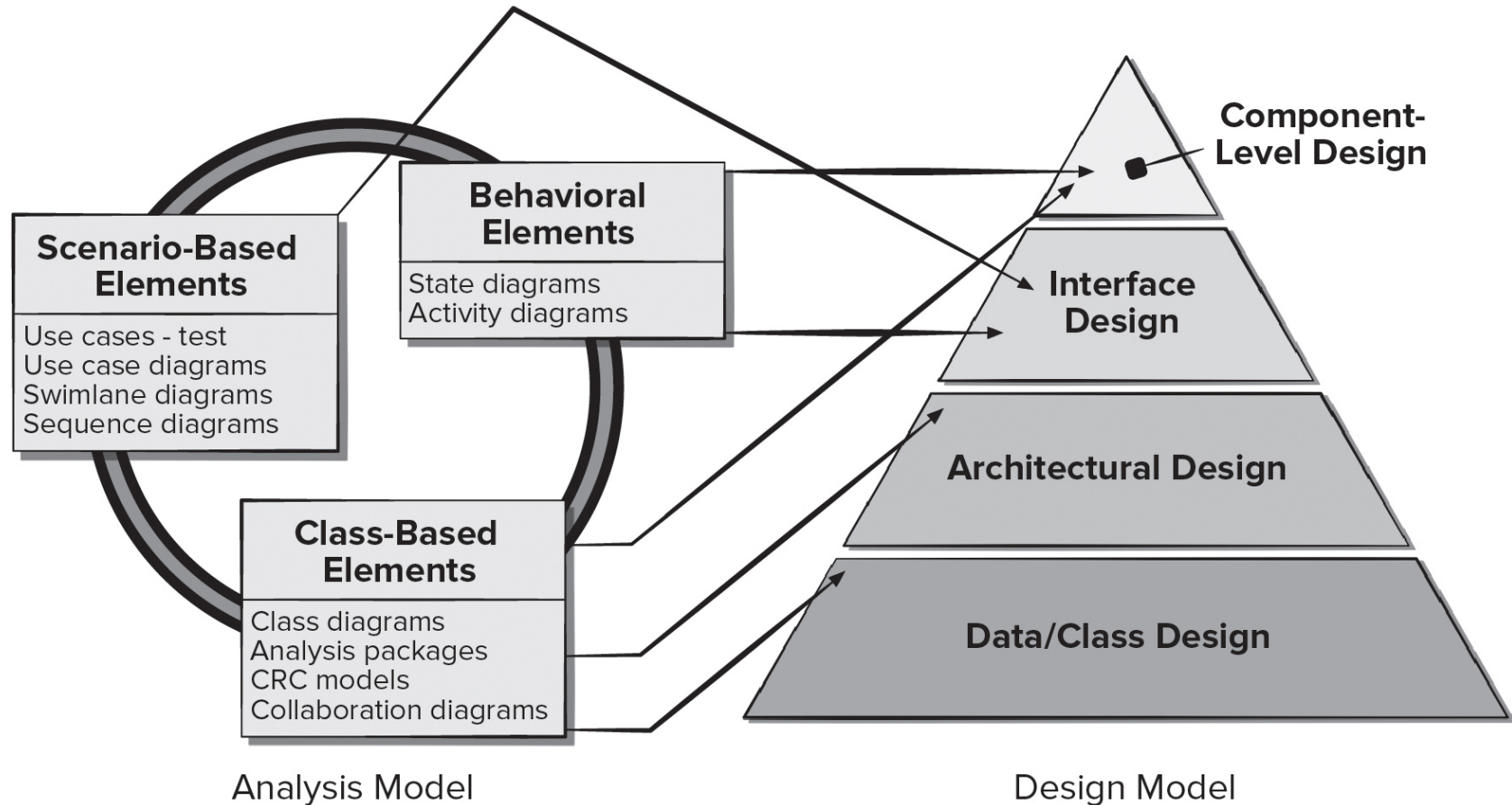
- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function and behavior of system to be built?
- Has the requirements model been “partitioned” in a way that exposes progressively more detailed information about the system?
- Have requirements patterns been used to simplify the requirements model. Have all patterns been properly validated? Are all patterns consistent with customer requirements?

Software Engineering Design

- Data/Class design – transforms analysis classes into implementation classes and data structures.
- Architectural design – defines relationships among the major software structural elements.
- Interface design – defines how software elements, hardware elements, and end-users communicate.
- Component-level design – transforms structural elements into procedural descriptions of software components.

Mapping Requirements Model to Design Model

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



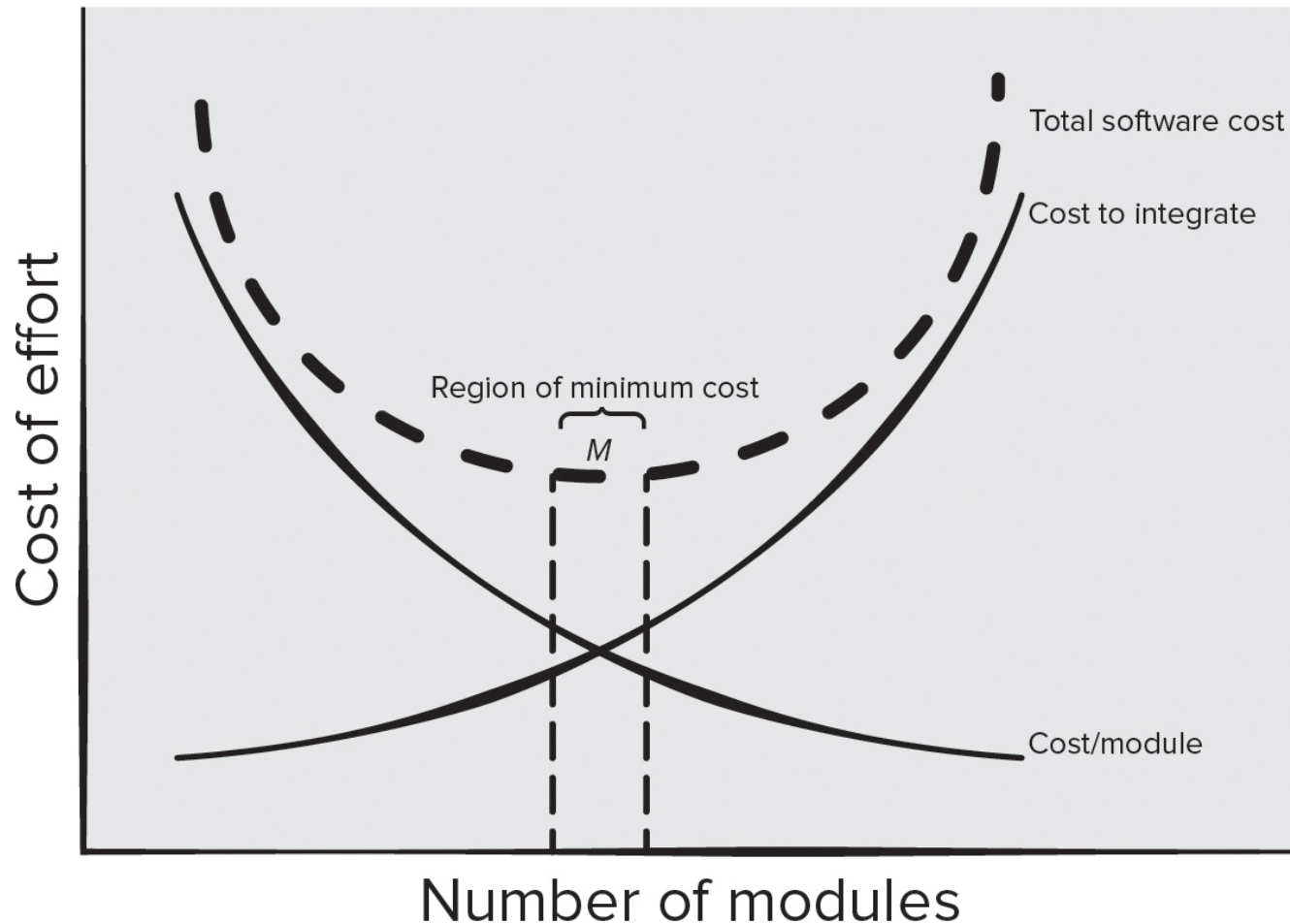
[Access the text alternative for slide images.](#)

Design Concepts ₁

- Abstraction – data (named collection of data describing data object), procedural (name sequence of instructions with specific and limited function).
- Architecture - overall structure or organization of software components, ways components interact, and structure of data used by components.
- Design Patterns - describe a design structure that solves a well-defined design problem within a specific context.
- Separation of concerns - any complex problem can be more easily handled if it is subdivided into pieces.
- Modularity—compartmentalization of data and function.

Modularity and Software Cost

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Design Concepts ²

- Information Hiding - controlled interfaces which define and enforces access to component procedural detail and any local data structure used by the component.
- Functional independence - single-minded (high cohesion) components with aversion to excessive interaction with other components (low coupling).
- Stepwise Refinement – incremental elaboration of detail for all abstractions.
- Refactoring—a reorganization technique that simplifies the design without changing functionality.
- Design Classes—provide design detail that will enable analysis classes to be implemented.

What is Software Architecture?

The architecture is not the operational software, it is a representation that enables a software engineer to:

1. Analyze the effectiveness of the design in meeting its stated requirements,
2. Consider architectural alternatives at a stage when making design changes is still relatively easy, and
3. Reduce the risks associated with the construction of the software.

Why is Software Architecture Important?

- Software architecture provides a representation that facilitates communication among all stakeholders interested in the development of a computer-based system.
- Architecture highlights early design decisions that will have a profound impact on all software engineering work that follows.
- Architecture constitutes a relatively small, intellectually graspable mode of how the system is structured and how its components work together.

Basic Component Design Principles

- **Open-Closed Principle (OCP).** *“A module [component] should be open for extension but closed for modification.”*
- **Liskov Substitution Principle (LSP).** *“Subclasses should be substitutable for their base classes.”*
- **Dependency Inversion Principle (DIP).** *“Depend on abstractions. Do not depend on concretions.”*
- **Interface Segregation Principle (ISP).** *“Many client-specific interfaces are better than one general purpose interface.”*
- **Release Reuse Equivalency Principle (REP).** *“The granule of reuse is the granule of release.”*
- **Common Closure Principle (CCP).** *“Classes that change together belong together.”*
- **Common Reuse Principle (CRP).** *“Classes that aren’t reused together should not be grouped together.”*

Component-Level Design Guidelines

- **Components** - Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model.
- **Interfaces** - provide important information about communication and collaboration (as well as helping us to achieve the OPC).
- **Dependencies and Inheritance** – For readability, it is a good idea to model dependencies from left to right and inheritance from bottom (derived classes) to top (base classes).

Cohesion

Traditional view - the “single-mindedness” of a module.

Object-Oriented view - *cohesion* implies that a component encapsulates only attributes and operations that are closely related to one another and the component itself.

Levels of cohesion:

- Functional - module performs one and only one computation.
- Layer - occurs when a higher layer accesses the services of a lower layer, but lower layers do not access higher layers.
- Communicational - All operations that access the same data are defined within one class.

Coupling

Traditional view - degree to which a component is connected to other components and to the external world.

Object-Oriented view - qualitative measure of the degree to which classes are connected to one another.

Levels of coupling.

- Content - occurs when one component “surreptitiously modifies data that is internal to another component.
- Control – occurs when control flags are passed to components to request alternate behaviors when invoked.
- External - occurs when a component communicates or collaborates with infrastructure components.

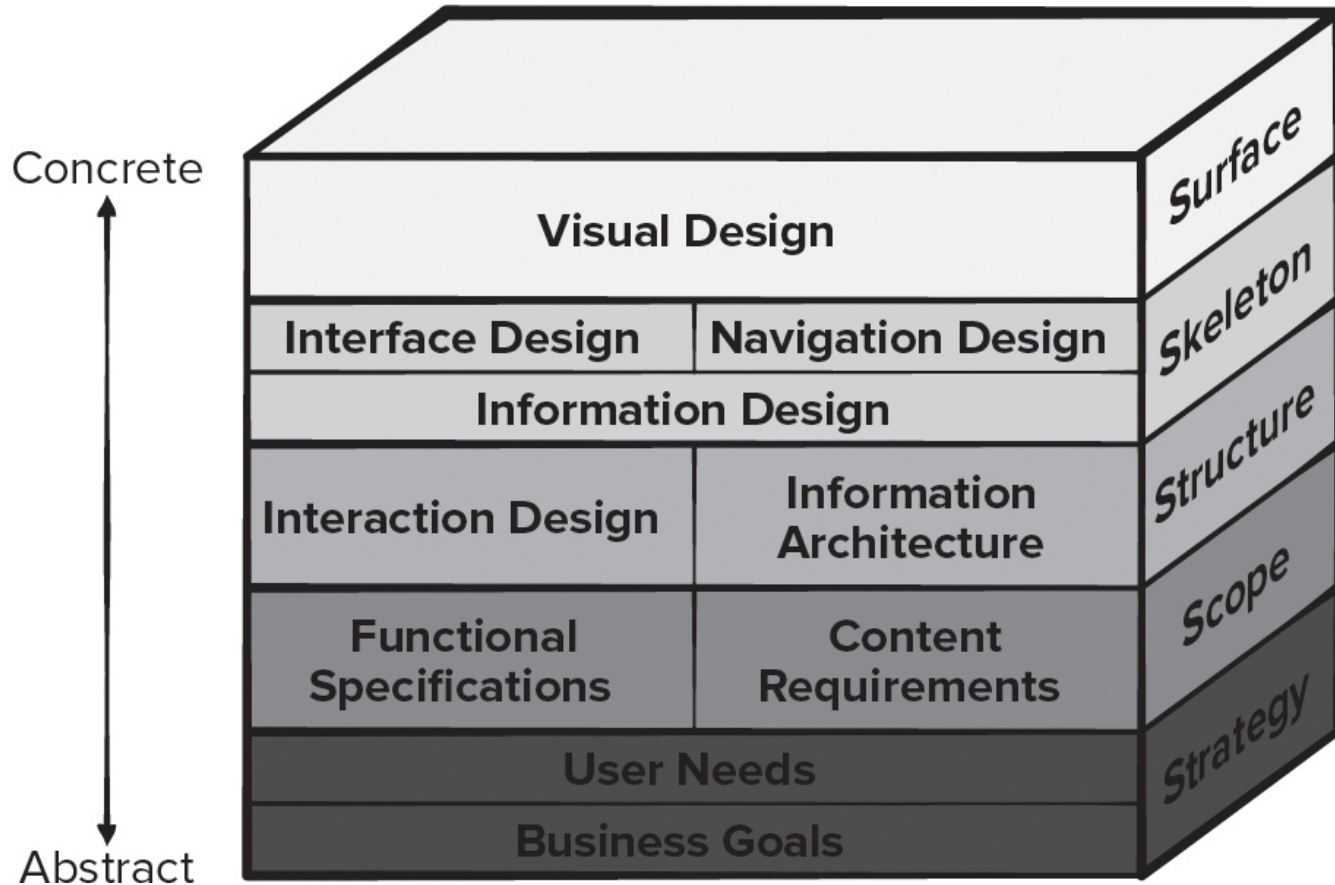
User Experience Design Elements ¹

User experience design tries to ensure that no aspect of your software appears the final product without the explicit decision of stakeholders to include it.

- **Strategy.** Identifies user needs and customer business goals that form the basis for all UX design work.
- **Scope.** Includes both the functional and content requirements needed to realize a feature set consistent with the project strategy.
- **Structure.** Consists of the interaction design [For example, how the system reacts in response to user action] and information architecture.
- **Skeleton.** Comprised of three components: information design, interface design, navigation design.
- **Surface.** Presents visual design or the appearance of the finished project to its users.

User Experience Design Elements ²

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Golden Rule 1: Place User in Control

- Define interaction modes in a way that does not force a user into unnecessary or undesired actions.
- Provide for flexible interaction.
- Allow user interaction to be interruptible and undoable.
- Streamline interaction as skill levels advance and allow the interaction to be customized.
- Hide technical internals from the casual user.
- Design for direct interaction with screen objects.

Golden Rule 2: Reduce User's Memory Load

- Reduce demand on short-term memory.
- Establish meaningful defaults.
- Define shortcuts that are intuitive.
- The visual layout of the interface should be based on a real-world metaphor.
- Disclose information in a progressive fashion.

Golden Rule 3: Make Interface Consistent

- Allow the user to put the current task into a meaningful context.
- Maintain consistency across a family of applications.
- If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so.

Context-Aware Apps

- Context allows the creation of apps based on the location of the mobile device and the device functionality to be delivered.
- Context helps tailor personal computer apps for mobile devices.
- Mobile computing merges real and virtual worlds by allowing devices to be aware of other objects and its surroundings.
- Device must detect the presence and identity of a user, as well as the attributes of the context that are relevant for that user.
- Extracting relevant context information from several sensors is challenging (for example: noise, miscalibration, wear and tear, weather).
- Event-based communication is preferable to the management of continuous streams of high-abstraction-level data in context-aware applications.

Reviews

What are they?

- A meeting conducted by technical people.
- A technical assessment of a work product created during the software engineering process.
- A software quality assurance mechanism.
- A training ground.

What they are not!

- A project summary or progress assessment.
- A meeting intended solely to impart information.
- A mechanism for political or personal reprisal!

Formal Technical Reviews

The objectives of an FTR (*walkthrough* or *inspection*) are:

1. To uncover errors in function, logic, or implementation for any representation of the software.
2. To verify that the software under review meets its requirements.
3. To ensure that the software has been represented according to predefined standards.
4. To achieve software that is developed in a uniform manner.
5. To make projects more manageable.

Review Guidelines

- Review the product, not the producer.
- Set an agenda and maintain it.
- Limit debate and rebuttal.
- Enunciate problem areas, but don't try to solve every problem noted.
- Take written notes.
- Limit the number of participants and insist upon advance preparation.
- Develop a checklist for each product that is likely to be reviewed.
- Allocate resources and schedule time for FTRs.
- Conduct meaningful training for all reviewers.
- Review your early reviews.

Elements of SQA

- Standards.
- Reviews and Audits.
- Testing.
- Error/defect collection and analysis.
- Change management.
- Education.
- Vendor management.
- Security management.
- Safety.
- Risk management.

Role of SQA Group 1

Prepares an SQA plan for a project which identifies:

- Evaluations to be performed.
- Audits and reviews to be performed.
- Standards that are applicable to the project.
- Procedures for error reporting and tracking.
- Documents to be produced by the SQA group.
- Amount of feedback provided to the software project team.

Participates in the development of the project's software process description.

- Reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (for example, ISO-9001), and other parts of the software project plan.

Role of SQA Group 2

Reviews software engineering activities to verify compliance with the defined software process.

- Identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

Audits designated software work products to verify compliance with those defined as part of the software process.

- Reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made.
- Periodically reports the results of its work to the project manager.

Ensures that deviations in software work and work products are documented and handled according to a documented procedure.

Records any noncompliance and reports to senior management.

- Noncompliance items are tracked until they are resolved.

Strategic Approach to Testing

- You should conduct effective technical reviews this can eliminate many errors before testing begins.
- Testing begins at the component level and works "outward" toward the integration of the entire system.
- Different testing techniques are appropriate for different software engineering approaches and at different points in time.
- Testing is conducted by the developer of the software and (for large projects) an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

Testing the Big Picture

- *Unit testing* begins at the center of the spiral and concentrates on each unit (for example, component, class, or content object) as they are implemented in source code.
- Testing progresses to *integration testing*, where the focus is on design and the construction of the software architecture.
Taking another turn outward on the spiral.
- *Validation testing*, is where requirements established as part of requirements modeling are validated against the software that has been constructed.
- In *system testing*, the software and other system elements are tested as a whole.

White Box Testing

Using white-box testing methods, you can derive test cases that:

1. Guarantee that all independent paths within a module have been exercised at least once.
2. Exercise all logical decisions on their true and false sides.
3. Execute all loops at their boundaries and within their operational bounds.
4. Exercise internal data structures to ensure their validity.

Black Box Testing ¹

Black-box (functional) testing attempts to find errors in the following categories:

1. Incorrect or missing functions.
2. Interface errors.
3. Errors in data structures or external database access.
4. Behavior or performance errors.
5. Initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, black-box testing tends to be applied during later stages of testing.

Black Box Testing ²

Black-box test cases are created to answer questions like:

- How is functional validity tested?
- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation?

Role of Scaffolding

- Components are not stand-alone program some type of *scaffolding* is required to create a testing framework.
- As part of this framework, driver and/or stub software must often be developed for each unit test.
- A *driver* is nothing more than a “main program” that accepts test-case data, passes such data to the component (to be tested), and prints relevant results.
- *Stubs* (dummy subprogram) serve to replace modules invoked by the component to be tested.
- A stub uses the module’s interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing.

Factors Affecting Software Team Structure

- **Difficulty** of the problem to be solved.
- **Size** of the resultant program(s) in lines of code or function points.
- **Team lifetimes** - time that the team will stay together.
- Degree to which the problem can be **modularized**.
- Required **quality** and **reliability** of the system to be built.
- Rigidity of the **delivery date**.
- **Communication** (degree of sociability) required.

Team Toxicity Factors

- **Frenzied work atmosphere** team members waste energy and lose focus on work objectives.
- **High frustration** caused by personal, business, or technological factors causing team member friction.
- **Fragmented or poorly coordinated procedures** poorly defined or improperly chosen process model.
- **Unclear definition of roles** resulting in lack of accountability and resultant finger-pointing.
- **Continuous and repeated exposure to failure** leads to a loss of confidence and a lowering of morale.

Agile Teams

Team members must have trust in one another.

The distribution of skills must be appropriate to the problem.

Mavericks may have to be excluded from the team, if team cohesiveness is to be maintained.

Team is “self-organizing.”

- An adaptive team structure.
- Planning is kept to a minimum.
- Team select its own approach constrained by business requirements and organizational standards.

Reactive Risk Management

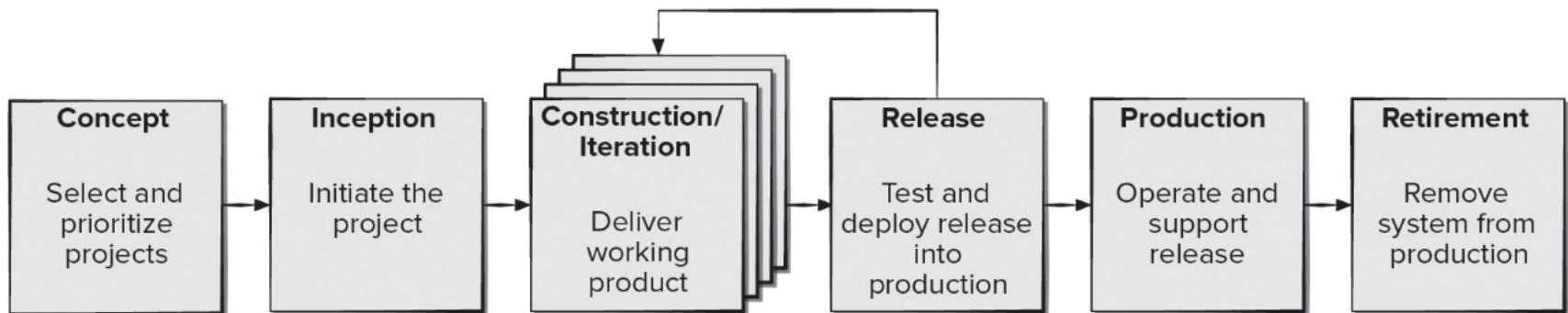
- Project team reacts to risks when they occur.
- Mitigation—plan for additional resources in anticipation of fire fighting.
- Fix on failure—resource are found and applied when the risk strikes.
- Crisis management—failure does not respond to applied resources and project is in jeopardy.

Proactive Risk Management

- Potential risks are identified, their probability and impact are assessed, and they are ranked by importance.
- Software team establishes a plan for managing risk.
- Primary objective is to avoid risk, but because not all risks can be avoided.
- Team works to develop a contingency plans that will enable it to respond in a controlled and effective manner.
- Proactive risk management is a software engineering tools that can be used to reduce technical debt.

Prototype Evolution Process Model

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

期末作业论文提交

Overleaf学术论文模板：<https://www.overleaf.com/latex/templates/acm-conference-proceedings-primary-article-template/wbvngghjbzwpc>

论文请于1月24日前提交给助教，每个小组一份。

提交内容为：overleaf project的源代码压缩包+导出后的pdf文件